

vsomeip DTLS Requirements Specification

System and software requirements, architecture decisions, implementation steps and open issues for opt-in DTLS 1.2 on unicast SOME/IP over UDP.

DOCUMENT ID	VSOMEIP-DTLS-SRS	VERSION	0.3 · Draft for review
AUTHOR	duonghvu	DATE	17 Sep 2026
REVIEWER	To be assigned	APPROVER	To be assigned
BRANCH	duonghvu/vsomeip · test/dtls-udp	BASE	COVESA vsomeip 3.7.5 (7875a1cb)
REPLACES	vsomeip_DTLS_UDP_3_7_2_Plan.md rev 0.2 (Markdown plan; kept outside the repository)		

Revision history

REV	DATE	AUTHOR	CHANGE
0.1	07 May 2026	duonghvu	Initial plan pinned to vsomeip 3.7.2: ADRs, file inventory, phases.
0.2	07 May 2026	duonghvu	Implementation pass on <code>feature/dtls-udp</code> : backend, session, endpoint classes, simulation. OpenSSL minimum raised to 3.0.
0.3	17 Sep 2026	duonghvu	Converted to a requirements specification with IDs, status and verification. Test branch rebased by merge onto vsomeip 3.7.5. Code-review findings added.

1 Scope

This document specifies native DTLS 1.2 protection for unicast SOME/IP over UDP inside the vsomeip library. A service opts in through configuration; application source code does not change. SOME/IP service discovery multicast, TCP transport, local IPC and the SOME/IP wire format are out of scope and stay unchanged.

Out of scope

- DTLS 1.3
- DTLS or signing for SD multicast
- HSM integration (possible later behind `dtls::backend`)
- Automatic certificate rotation
- Post-quantum cipher suites

Terms

- **DTLS**: Datagram TLS, RFC 6347 (1.2)
- **SD**: SOME/IP Service Discovery
- **EARS**: Easy Approach to Requirements Syntax
- **ADR**: Architecture Decision Record
- **Peer**: a remote (IP address, UDP port) pair

Requirement conventions

Requirements use EARS wording with “shall”. **Status** is the state on `test/dtls-udp` as of the document date. **Verification** names the planned method: UT unit test, CT component test, IT integration test (multi-ECU, fake sockets), QT qualification test (real processes and sockets), ST system test (bench), RV review or inspection.

Implemented in code and exercised **Partial** code exists but incomplete, unwired or unverified

Not met code contradicts the requirement **Open** not started

2 Constraints

ID	CONSTRAINT	SOURCE
CON-01	The existing <code>udp_server_endpoint_impl.*</code> and <code>udp_client_endpoint_impl.*</code> shall not be modified; DTLS lives in new classes alongside them.	ADR-001
CON-02	Sources under <code>implementation/service_discovery/</code> shall not be modified.	ADR-005
CON-03	The SOME/IP wire format and the public API in <code>interface/vsomeip/</code> shall not change.	Plan §4
CON-04	Existing JSON keys shall not be renamed or removed; additions only.	ADR-004
CON-05	New code shall not depend on <code>RS_DIAGNOSIS</code> .	3.7.2 deprecation
CON-06	C++17, <code>vsomeip_v3</code> namespace, MPL-2.0 header, clang-format per CONTRIBUTING.md, no raw <code>new / delete</code> , logging only through <code>VSOMEIP_*</code> .	Project conventions
CON-07	Files outside the plan's file inventory shall not be changed without recorded operator approval.	Plan §4

3 System requirements

ID	REQUIREMENT	VERIFICATION	STATUS
SYS-DTLS-001	The ECU shall protect unicast SOME/IP UDP traffic of configured services with mutually authenticated DTLS 1.2.	ST, QT	Partial
SYS-DTLS-002	The ECU shall enable DTLS per service through configuration alone, without application source changes.	QT	Open
SYS-DTLS-003	SOME/IP-SD multicast shall remain plaintext and interoperable with ECUs without DTLS.	QT, ST	Implemented
SYS-DTLS-004	If a peer presents a certificate not issued by the configured trust anchor, the ECU shall refuse communication with that peer.	QT, ST	Partial
SYS-DTLS-005	The ECU shall bound the resources an unauthenticated source can consume during handshakes.	QT, ST	Not met
SYS-DTLS-006	Services not configured for DTLS, and builds without DTLS support, shall behave as in the upstream vsomeip release they are based on.	QT	Partial

4 Software requirements

4.1 Configuration

ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
SWR-DTLS-001	The configuration module shall parse an optional global <code>dtls</code> object with the keys <code>enable</code> , <code>version</code> , <code>cipher_list</code> , <code>certificate</code> , <code>private_key</code> , <code>ca_bundle</code> , <code>verify_peer</code> , <code>handshake_timeout_ms</code> and <code>cookie_exchange</code> , applying the defaults in Annex A for absent keys.	SYS-002	UT	Partial Parser written; not called by configuration_impl
SWR-DTLS-002	If <code>dtls.enable</code> is true and <code>certificate</code> or <code>private_key</code> is empty, the configuration module shall treat DTLS as disabled and log a warning.	SYS-001	UT	Partial Disables; warning not wired
SWR-DTLS-003	If <code>dtls.enable</code> and <code>verify_peer</code> are true and <code>ca_bundle</code> is empty, the configuration module shall treat DTLS as disabled and log a warning.	SYS-004	UT	Partial Disables; warning not wired
SWR-DTLS-004	The configuration module shall accept a per-service key <code>unreliable_secure</code> holding a UDP port number.	SYS-002	UT	Open
SWR-DTLS-005	When a service declares both <code>unreliable</code> and <code>unreliable_secure</code> , the configuration module shall log a warning.	SYS-002	UT	Open
SWR-DTLS-006	If <code>dtls.version</code> is not "1.2", the configuration module shall log an error and treat DTLS as disabled.	SYS-001	UT	Open Proposed

4.2 Endpoint creation

ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
SWR-DTLS-010	When <code>dtls.enable</code> is true and an offered service declares <code>unreliable_secure</code> , the endpoint manager shall create a DTLS UDP server endpoint bound to that port.	SYS-001, 002	IT	Open
SWR-DTLS-011	When the local configuration marks a remote service instance as <code>unreliable_secure</code> , the endpoint manager shall create a DTLS UDP client endpoint towards that instance's unreliable port.	SYS-001, 002	IT	Open Depends on OI-01
SWR-DTLS-012	When DTLS is disabled or invalid, the endpoint manager shall not create a plaintext endpoint on an <code>unreliable_secure</code> port.	SYS-001	IT	Open Depends on OI-02
SWR-DTLS-013	The routing manager shall mark a DTLS service's <code>serviceinfo</code> during the preparation stage, before the service is offered.	SYS-002	IT	Open
SWR-DTLS-014	DTLS endpoints shall not join multicast groups.	SYS-003	CT, RV	Implemented

4.3 Handshake and authentication

ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
SWR-DTLS-020	The DTLS endpoints shall negotiate DTLS 1.2 only.	SYS-001	UT, QT	Implemented Min and max protocol pinned
SWR-DTLS-021	When <code>verify_peer</code> is true, the DTLS endpoints shall require a peer certificate and verify that it chains to <code>ca_bundle</code> .	SYS-001, 004	UT, QT	Implemented Simulation only
SWR-DTLS-022	If peer verification fails, the endpoint shall abort the handshake, release the session and log the peer address and failure reason.	SYS-004	UT, CT	Partial Server only; not compiled
SWR-DTLS-023	When <code>cookie_exchange</code> is true, the server shall issue cookies derived from a keyed MAC over the client address and shall allocate handshake state only after a valid cookie is returned.	SYS-005	UT, QT	Not met Static cookie; session allocated first
SWR-DTLS-024	If the random generator fails while creating the cookie secret, the server shall refuse handshakes and log an error.	SYS-005	UT	Not met Falls back to predictable bytes
SWR-DTLS-025	The DTLS endpoints shall restrict cipher suites to <code>cipher_list</code> , default <code>ECDHE-ECDSA-AES128-GCM-SHA256</code> .	SYS-001	UT	Implemented
SWR-DTLS-026	If a handshake does not complete within <code>handshake_timeout_ms</code> , the endpoint shall abort it, release the session and log a warning.	SYS-005	UT	Not met Value parsed, never used

ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
SWR-DTLS-027	While a handshake is in progress, the session shall retransmit flights using the DTLS timer and back-off.	SYS-001	UT	Implemented

4.4 Data path

ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
SWR-DTLS-030	The server endpoint shall keep one DTLS session per peer.	SYS-001	CT	Partial Not compiled
SWR-DTLS-031	After decryption, the endpoint shall deliver each SOME/IP message through the same receive path as the plain UDP endpoint.	SYS-002	CT, IT	Partial Not compiled
SWR-DTLS-032	The SOME/IP messages carried inside DTLS records shall be byte-identical to those sent over plain UDP.	SYS-003, 006	QT	Partial
SWR-DTLS-033	While the handshake is in progress, the client endpoint shall hold outgoing messages and send them in order once it completes.	SYS-002	CT	Partial Retry not verified
SWR-DTLS-034	When an endpoint stops, it shall send <code>close_notify</code> to each established peer and release all sessions.	SYS-001	CT	Partial Server; not compiled
SWR-DTLS-035	The endpoints shall not emit DTLS records larger than the configured path MTU; larger SOME/IP messages shall use SOME/IP-TP.	SYS-001	QT, ST	Partial MTU fixed at 1500

4.5 Robustness, errors and logging

ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
SWR-DTLS-040	The server endpoint shall limit concurrent sessions to a configurable maximum and evict sessions idle longer than a configurable time.	SYS-005	CT, QT	Not met Unbounded map
SWR-DTLS-041	Endpoints and sessions shall hold no reference cycles; both shall be released after stop.	SYS-006	CT (ASan/LSan)	Not met shared_from_this in callbacks
SWR-DTLS-042	When a peer restarts, the client endpoint shall establish a new session without application involvement.	SYS-001	IT, ST	Open
SWR-DTLS-050	The crypto backend shall report failures as result codes; no exception shall cross an endpoint boundary.	SYS-006	UT, RV	Implemented
SWR-DTLS-051	The library shall not log private keys, session secrets or decrypted payload bytes.	SYS-001	RV	Partial Needs review

ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
4.6 Build and compatibility				
SWR-DTLS-052	If a handshake fails, the log entry shall contain the peer address and, if available, the peer certificate subject.	SYS-004	CT	Partial
ID	REQUIREMENT	TRACES TO	VERIFICATION	STATUS
SWR-DTLS-060	The build shall provide a CMake option <code>ENABLE_DTLS</code> , default OFF.	SYS-006	RV	Implemented
SWR-DTLS-061	When <code>ENABLE_DTLS</code> is OFF, the library shall pass the upstream test suite unchanged.	SYS-006	QT	Partial
SWR-DTLS-062	When <code>ENABLE_DTLS</code> is ON, the library shall build, and configuration shall fail with a clear message if OpenSSL 3 is not found.	SYS-001	QT	Not met
SWR-DTLS-063	Only the backend implementation shall include crypto library headers.	ADR-002	RV	Implemented
SWR-DTLS-064	Cipher suites in configuration shall use IANA names, translated by each backend.	ADR-002	UT	Open Proposed
SWR-DTLS-070	DTLS endpoints shall work in classic routing mode. If a conflict with hybrid mode is found, DTLS shall be disabled in hybrid mode with a warning instead of changing hybrid-mode behaviour.	ADR-006	IT	Open

5 Architecture decisions

ADR	DECISION	RATIONALE
ADR-001	New <code>dtls_udp_server/client_endpoint_impl</code> classes next to the UDP endpoints; existing endpoints untouched.	Small blast radius; one CMake switch removes it; reviewable upstream.
ADR-002	OpenSSL 3 first, behind a <code>dtls::backend</code> interface.	Most mature DTLS stack; allows Mbed TLS or HSM-backed crypto later.
ADR-003	One DTLS session per peer (IP, port); the client endpoint holds one session.	Matches SOME/IP unicast usage.
ADR-004	Opt-in per service with <code>unreliable_secure</code> ; <code>unreliable</code> unchanged.	Backward compatible; staged rollout; no application changes.
ADR-005	SD multicast stays plaintext.	No per-peer session exists for multicast; SD signing is future work.
ADR-006	Parity with hybrid mode; on conflict, disable DTLS in hybrid mode.	Hybrid mode is new upstream behaviour and must not be destabilised.

6 Implementation steps

6.1 Phase status

PHASE	GOAL	EXIT CRITERION	STATUS
P1 Baseline	Build upstream on Linux, record test baseline, observe hybrid mode	Library builds; baseline test list and hybrid-mode behaviour recorded	Open Only macOS used so far; now on 3.7.5
P2 Build switch	<code>ENABLE_DTLS</code> option and source skeletons	Builds with ON and OFF; OFF unchanged	Partial ON fails to compile
P3 Crypto core	OpenSSL backend and per-peer session	Loopback handshake and 64-byte payload in a unit test	Partial Proven in simulation; no ctest unit test
P4 Server endpoint	Accept, handshake, decrypt, deliver	Registered application receives a message over DTLS	Partial Written, not compiled
P5 Client endpoint	Connect, handshake, request/response	One request and response over DTLS	Partial Written, not compiled
P6 Wiring	Config, factory and routing integration; <code>hello_world_dtls</code>	<code>hello_world</code> runs over DTLS through JSON only	Open
P7 Robustness and docs	Timeouts, revocation, MTU, resumption, <code>close_notify</code> , user docs	All tests green; docs merged; integration package archived	Open

6.2 Ordered next steps

1. **Record the Linux baseline on 3.7.5.** Build in the repository Dockerfile (add `libssl-dev`) with `ENABLE_DTLS=OFF`; run `ctest -R '^unit_'` and the fake-socket tests; save the results.
P1 · SWR-DTLS-061
2. **Make the endpoints compile against 3.7.5.** Create sockets through the injected `abstract_socket_factory`; update `tp_segmentation_enabled` to take `service_instance_t`; implement any new base-class hooks.
P2, P4, P5 · SWR-DTLS-062
3. **Correct the cookie design.** Pass the peer address into `create_server_session`; compute the cookie as an HMAC of the address with a rotating random secret; refuse handshakes if the random generator fails; don't allocate a session before the cookie is verified.
SWR-DTLS-023, 024
4. **Bound session state.** Enforce `handshake_timeout_ms`, cap sessions, evict idle ones; make MTU configurable.
SWR-DTLS-026, 035, 040
5. **Remove reference cycles.** Capture `weak_ptr` in session callbacks.
SWR-DTLS-041
6. **Add unit tests** `unit_dtls_backend`, `unit_dtls_session`, `unit_dtls_configuration` in `test/unit_tests/dtls_tests/`, including wrong CA, expired certificate, loss and reordering.
P3 exit · SWR-DTLS-001...027
7. **Wire the configuration.** `configuration_impl::load_dtls`, per-service `unreliable_secure`, getters and warnings.
SWR-DTLS-001...006
8. **Wire endpoint creation and routing.** Factory branch in `endpoint_manager_impl`; `serviceinfo` mark in `routing_manager_impl`; decide OI-01 and OI-02 first.
SWR-DTLS-010...013
9. **Add component and multi-ECU tests** with fake sockets, including hybrid mode, and the `hello_world_dtls` example.
P6 exit · SWR-DTLS-030...034, 070
0. **Add qualification tests** on two containers: interop with `openssl s_client/s_server`, tcpdump check of SD and service ports, 1000-handshake stress, sanitizers, ON and OFF suites.
P7 · SYS-DTLS-001...006
1. **Document and publish.** Update `vsomeipConfiguration.md` and the user guide; add the CI matrix for `ENABLE_DTLS` ON and OFF; request community review.
P7
2. **Bench validation** on target ECUs: automotive Ethernet MTU, OEM PKI, revocation, peer reboot, CPU and latency budget.
SYS.5

7 Verification plan

METHOD	LEVEL	ENVIRONMENT	LOCATION	ENTRY → EXIT
UT	SWE.4 unit	In-process, no sockets; macOS, Linux, CI	test/unit_tests/dtls_tests/	Code compiles → all pass, ASan/UBSan clean
CT	SWE.5 component	Fake socket factory; Linux, CI	test/unit_tests/endpoint_tests/	UT green → endpoint contract met, no leaks
IT	SWE.5 integration	Multiple ECUs in one process with fault injection; Linux, CI	test/network_tests/fake_socket_tests/	CT green → offer, subscribe and request over DTLS; hybrid mode checked
QT	SWE.6 qualification	Two containers, real sockets; nightly	test/network_tests/dtls_tests/ , zuul/network-tests	IT green → every SWR has a passing test; ON and OFF suites green
ST	SYS.5 system	Target ECUs on bench	Outside repository	QT green → every SYS acceptance criterion met
RV	All	Code review, static analysis, MISRA C++ baseline	Pull requests	Security reviewer signs off changes to the backend and session

8 Open issues

ID	QUESTION	OPTIONS	PROPOSAL
OI-01	How does a client learn that a remote service instance requires DTLS?	(a) local config marks remote instances <code>unreliable_secure</code> ; (b) new SD endpoint option	(a): (b) changes SD, which CON-02 forbids
OI-02	What happens when <code>unreliable_secure</code> is set but DTLS is disabled or invalid?	(a) fail closed, don't offer; (b) fall back to plaintext	(a) with an error log; needs security owner decision
OI-03	How does the server send events to subscribers without an established session?	(a) queue until the client handshakes; (b) client connects on subscribe	(b), with (a) as a bounded fallback
OI-04	Should the peer certificate identity be bound to the expected ECU per service?	(a) chain check only; (b) also match subject or SAN	(b) as an optional per-service setting
OI-05	Certificate revocation scope	(a) CRL file; (b) OCSP stub; (c) out of scope	(a) for the first release

ID	QUESTION	OPTIONS	PROPOSAL
0I-06	Baseline OpenSSL version	3.0 (end of life Sep 2026) or 3.5 LTS	Document 3.5 LTS; keep 3.0 as build minimum

A Annex A: dtls configuration keys

KEY	TYPE	DEFAULT	NOTES
enable	bool string	"false"	Master switch
version	string	"1.2"	Only 1.2 supported
cipher_list	string	"ECDHE-ECDSA-AES128-GCM-SHA256"	Currently OpenSSL syntax (see SWR-DTLS-064)
certificate	path	none	PEM, ECU identity
private_key	path	none	PEM; may be HSM-backed in other backends
ca_bundle	path	none	PEM, trusted issuers
verify_peer	bool string	"true"	Mutual authentication; "false" needs a written justification
handshake_timeout_ms	uint	5000	Handshake budget
cookie_exchange	bool string	"true"	HelloVerifyRequest (DoS mitigation)
unreliable_secure	port (per service)	none	DTLS UDP port for the service

B References

- RFC 6347, Datagram Transport Layer Security Version 1.2
- RFC 9146, Connection Identifier for DTLS 1.2
- COVESA vsomeip 3.7.5 source and CONTRIBUTING.md
- vsomeip_DTLS_Summary.pdf (VSOMEIP-DTLS-SUM)
- OpenSSL 3 documentation, DTLS_method(3)
- Automotive SPICE PAM 3.1: SYS.2, SWE.1–SWE.6, SYS.5

C Approval

ROLE	NAME	DATE	SIGNATURE
Author	duonghvu	17 Sep 2026	
Technical reviewer			
Security reviewer			
Approver			

Author: duonghvu · Drafted with Claude Code assistance · Status values reflect a code reading of test/dtls-udp on 17 Sep 2026, not test results.